Trabalho Final — Classificação de Risco de Crédito (German Credit)

Nov 8, 2026 · @aaaa

Universidade Federal de Uberlândia — Ciência de Dados I — Sistemas de Informação

Professora: Elaine Ribeiro Faria · Tema: Classificação · Base de dados nº 10: Statlog (German Credit Data)

Estudante	Matrícula
[Nome do estudante 1]	[matrícula]
[Nome do estudante 2]	[matrícula]
[Nome do estudante 3]	[matrícula]

I. Introdução e objetivos

Este trabalho compara quatro algoritmos de classificação supervisionada para prever se um solicitante de crédito será **bom** ou **mau** pagador, usando a base Statlog (German Credit Data). Com o protocolo adotado, o **Random Forest** teve o menor custo de erro no teste (0,530 por cliente, contra 0,700 de negar crédito a todos).

Problema e relevância. A concessão de crédito é uma decisão de risco: aprovar quem não paga gera prejuízo direto, e negar quem pagaria faz o banco perder um cliente. A própria base define que o primeiro erro custa **5 vezes** mais que o segundo. Modelos de *credit scoring* apoiam essa decisão de forma padronizada e auditável, e são uma das aplicações mais antigas de aprendizado de máquina no setor financeiro.

Objetivos.

1. Explorar a base e descrever seus atributos, a classe, o desbalanceamento e a qualidade dos dados.
2. Definir e justificar o pré-processamento a partir das características observadas.
3. Aplicar quatro algoritmos — MLP, KNN, Random Forest e Gradient Boosting — com ao menos duas parametrizações cada.
4. Compará-los de forma justa (mesma divisão dos dados, mesmas medidas) e recomendar um modelo considerando o custo assimétrico dos erros.

II. Descrição da base de dados e análise exploratória

A base tem **1000 clientes**, **20 atributos preditores** (7 numéricos e 13 categóricos) e uma classe binária com **70% de bons e 30% de maus pagadores**, sem valores ausentes nem duplicatas.

Fonte. UCI Machine Learning Repository, *Statlog (German Credit Data)*, id 144, doada pelo Prof. Hans Hofmann (Universität Hamburg, 1994): archive.ics.uci.edu/dataset/144. Usamos o arquivo `german.data`, que mantém os atributos categóricos originais (códigos A11, A12…), decodificados com o dicionário `german.doc`.

Atributos

Atributo	Significado	Tipo	Valores
checking_status	Situação da conta corrente	categórico ordinal*	< 0 DM; 0–200 DM; ≥ 200 DM; sem conta
duration	Duração do crédito (meses)	numérico	4 a 72
credit_history	Histórico de crédito	categórico nominal	5 categorias (de "sem créditos/todos pagos" a "conta crítica")
purpose	Finalidade do crédito	categórico nominal	10 categorias (carro novo, rádio/TV, móveis, educação…)
credit_amount	Valor do crédito (marcos alemães, DM)	numérico	250 a 18.424
savings_status	Poupança / títulos	categórico ordinal*	< 100; 100–500; 500–1000; ≥ 1000 DM; desconhecido
employment	Tempo no emprego atual	categórico ordinal	desempregado; < 1; 1–4; 4–7; ≥ 7 anos
installment_rate	Parcela em % da renda disponível	numérico discreto	1 a 4
personal_status	Estado civil e sexo	categórico nominal	4 categorias
other_parties	Outros devedores / fiadores	categórico nominal	nenhum; co-requerente; fiador

Atributo	Significado	Tipo	Valores
residence_since	Tempo na residência atual	numérico discreto	1 a 4
property	Bens	categórico nominal	imóvel; poupança/seguro; carro/outros; nenhum
age	Idade (anos)	numérico	19 a 75
other_installment_plans	Outros parcelamentos	categórico nominal	banco; lojas; nenhum
housing	Moradia	categórico nominal	aluguel; própria; gratuita
existing_credits	Nº de créditos neste banco	numérico discreto	1 a 4
job	Qualificação profissional	categórico ordinal	4 níveis
num_dependents	Nº de dependentes	numérico discreto	1 ou 2
own_telephone	Telefone em nome do cliente	categórico nominal	sim; não
foreign_worker	Trabalhador estrangeiro	categórico nominal	sim; não

* ordinal exceto a categoria "sem conta"/"desconhecido", que não tem posição natural na escala.

Atributo classe e desbalanceamento

A classe `credit_risk` vale 1 = bom e 2 = mau no arquivo original; recodificamos para 0 = bom e 1 = mau, tratando **mau como a classe positiva** (a de maior custo).

Classe	Clientes	%
Bom pagador	700	70%
Mau pagador	300	30%

A razão de 2,33 bons por mau é um desbalanceamento moderado. Um classificador que diz sempre "bom" acerta 70%, mas tem custo médio de 1,50 por cliente (5 × 300 / 1000); um que diz sempre "mau" custa 0,70. Por isso a acurácia sozinha não serve, e usamos recall

e F1 da classe mau, AUC e custo.

Qualidade dos dados

- **Ausentes, duplicatas e rótulos conflitantes:** nenhum.
- **Categorias documentadas que nunca aparecem:** finalidade "férias" (A47) e "mulher solteira" (A95). Por isso `personal_status` não separa completamente sexo e estado civil.
- **Categorias raras (< 2%):** finalidade "eletrodomésticos" (12), "outros" (12) e "requalificação" (9) — candidatas a agrupamento.
- **Possíveis inconsistências:** 45 clientes "desempregados" com emprego qualificado e 27 com histórico "sem créditos/todos pagos" e mais de um crédito. Ambos têm leitura plausível (qualificação de quem está sem emprego; a categoria A30 também significa "todos pagos"), então **mantivemos as linhas**.
- **Valores extremos:** há outliers por IQR em valor (72), duração (70) e idade (23), todos plausíveis (créditos altos e longos); não foram removidos.
- **Atributos sensíveis:** `personal_status` (sexo) e `foreign_worker` (96% "sim") levantam questões de justiça algorítmica.

Atributos numéricos

Atributo	Média	Mediana	Mín–máx	Assimetria	Média (bons)	Média (maus)
duration (meses)	20,9	18	4–72	1,09	19,2	24,9
credit_amount (DM)	3.271	2.320	250–18.424	1,95	2.985	3.938
age (anos)	35,6	33	19–75	1,02	36,2	34,0
installment_rate	2,97	3	1–4	−0,53	2,92	3,10
residence_since	2,85	3	1–4	−0,27	2,84	2,85
existing_credits	1,41	1	1–4	1,27	1,42	1,37
num_dependents	1,16	1	1–2	1,91	1,16	1,15

Maus pagadores pedem créditos mais longos (+5,7 meses em média) e maiores (+953 DM) e são um pouco mais jovens. Valor, duração e idade têm assimetria à direita, e as escalas são muito diferentes (idade até 75, valor até 18 mil). O único par numérico fortemente correlacionado é duração × valor (Spearman 0,62).

Atributos categóricos

Categorias com maior taxa de mau pagador (entre as com pelo menos 30 clientes; média geral = 30%):

Atributo	Categoria	Clientes	% de maus
credit_history	sem créditos / todos pagos	40	62,5%
credit_history	todos pagos neste banco	49	57,1%
checking_status	saldo < 0 DM	274	49,3%
purpose	educação	50	44,0%
other_parties	co-requerente	41	43,9%
property	desconhecido / nenhum	154	43,5%

No outro extremo, clientes **sem conta corrente** (394) têm só 11,7% de maus, e os com poupança ≥ 1000 DM, 12,5%.

Associação com a classe

Testamos cada atributo contra a classe (qui-quadrado para categóricos, Mann-Whitney para numéricos) e medimos a informação mútua. **15 dos 20 atributos** têm associação significativa a 5%; os mais fortes são situação da conta corrente (Cramér's V = 0,35), duração, histórico de crédito, idade, poupança e finalidade. Não têm associação significativa: `existing_credits`, `job`, `own_telephone`, `residence_since` e `num_dependents`. Nenhum atributo separa bem as classes sozinho, então o ganho deve vir da combinação de atributos.

III. Metodologia

Todos os algoritmos seguem o mesmo protocolo: mesma divisão dos dados, pré-processamento ajustado só no treino, limiar e hiperparâmetros escolhidos na validação e teste usado uma única vez.

Ferramentas

Python 3.12 com pandas 3.0, NumPy 2.4 e SciPy 1.17 (análise e testes estatísticos), scikit-learn 1.8 (pré-processamento, KNN, Random Forest, Gradient Boosting e métricas), PyTorch 2.13 (MLP) e Matplotlib 3.10 (gráficos). Cada algoritmo tem sua pasta com um script de treino e arquivos de configuração YAML; um módulo comum executa o protocolo de avaliação para todos, o que garante que a comparação seja feita nas mesmas condições.

Pré-processamento

Etapa	Justificativa (da análise exploratória)	Aplicada em
Nenhuma imputação ou remoção de linhas	Não há ausentes nem duplicatas; outliers e casos suspeitos são plausíveis	todos
Recodificar a classe para 0 = bom, 1 = mau	Mau é a classe de interesse (maior custo)	todos
One-hot encoding dos 13 categóricos	São nominais, ou ordinais com uma categoria fora da escala ("sem conta", "desconhecido")	todos
Agrupar categorias com < 2% do treino em "infrequente"	Finalidades raras (9 a 12 clientes) geram colunas quase vazias e ruído	todos
log(1 + x) em valor e duração	Forte assimetria à direita (1,95 e 1,09)	MLP, KNN
Padronização z-score dos numéricos	Escalas muito diferentes; a MLP converge melhor e o KNN depende de distâncias	MLP, KNN
Sem log e sem padronização	Árvores só comparam valores com limiares, então a escala não muda o resultado	Random Forest, Gradient Boosting
Peso da classe mau = n_bons / n_maus do treino (2,33)	Compensa o desbalanceamento 70/30 durante o treino	MLP, Random Forest, Gradient Boosting (o KNN não aceita)

Após o one-hot, cada cliente é representado por 59 colunas. Escala, log, one-hot e agrupamento de categorias raras são **aprendidos só no conjunto de treino** e apenas aplicados a validação e teste, para evitar vazamento de dados.

Estratégia de avaliação

1. **Divisão:** as 1000 linhas são sorteadas em treino (600), validação (200) e teste (200), de forma estratificada (30% de maus em cada conjunto) e com semente fixa. **Todos os algoritmos usam exatamente as mesmas linhas** em cada conjunto; o código confere isso por um identificador da divisão.

2. **Treino:** cada modelo é ajustado no treino. O teste nunca é passado ao algoritmo. A MLP usa a validação para guardar o melhor checkpoint e parar o treino; os demais não usam a validação no ajuste.
3. **Limiar de decisão:** os modelos produzem P(mau); o cliente é classificado como mau quando P(mau) ≥ limiar. O limiar é escolhido **na validação**, entre 0,05 e 0,95, como o que minimiza o custo 5·FN + 1·FP.
4. **Escolha da parametrização:** para cada algoritmo, a melhor configuração é a de **menor custo médio na validação**.
5. **Teste:** a configuração escolhida é avaliada uma única vez no teste. São esses os números comparados entre algoritmos.

Como validação e teste têm só 200 clientes (cerca de 60 maus), repetimos a melhor configuração de cada algoritmo em mais 5 divisões aleatórias diferentes para medir a variabilidade dos resultados (seção IV).

Medidas de avaliação

Com a classe mau como positiva, VP = maus detectados, FN = maus aprovados, FP = bons negados e VN = bons aprovados. Usamos:

```
\begin{aligned}
\text{Acurácia} &= \frac{VP + VN}{VP + VN + FP + FN} \\[4pt]
\text{Precisão} &= \frac{VP}{VP + FP} \qquad \text{Recall} = \frac{VP}{VP + FN} \qquad \text{Especificidade} = \frac{VN}{VN + FP} \\[4pt]
F_1 &= \frac{2 \cdot \text{Precisão} \cdot \text{Recall}}{\text{Precisão} + \text{Recall}} \\[4pt]
\text{Custo médio} &= \frac{5 \cdot FN + 1 \cdot FP}{n}
\end{aligned}
```

Além dessas, a **AUC-ROC** é a área sob a curva que relaciona recall e taxa de falsos positivos em todos os limiares possíveis; ela mede a capacidade de ordenar maus acima de bons, independente do limiar. O **custo médio** é a medida principal, porque reflete a matriz de custo da própria base; recall, F1 e AUC complementam a análise. A acurácia é reportada, mas não é usada para escolher modelos.

Algoritmos

Escolhemos quatro algoritmos de famílias diferentes: uma rede neural (MLP), um método baseado em distância (KNN) e dois conjuntos de árvores (Random Forest e Gradient Boosting). **[Grupo: confirmar quais destes foram vistos em sala. O enunciado exige pelo menos dois não vistos, descritos com pseudocódigo; incluímos pseudocódigo para MLP, Random Forest e Gradient Boosting.]**

1. MLP — Perceptron Multicamadas (PyTorch)

Funcionamento. A rede recebe as 59 colunas de um cliente e as transforma em camadas sucessivas: cada neurônio calcula uma soma ponderada das saídas da camada anterior e aplica uma função de ativação não linear. A última camada produz um valor que a sigmoide converte em P(mau). Os pesos são ajustados por retropropagação do erro (backpropagation), minimizando a entropia cruzada binária com peso maior para a classe mau.

```
entrada: treino (X, y), validação (Xv, yv), peso da classe mau w
inicializa os pesos da rede aleatoriamente
melhor ← ∞;  sem_melhora ← 0
para cada época (até 300):
    embaralha o treino e divide em lotes de 32 clientes
    para cada lote:
        p ← sigmoide(rede(X_lote))                    # propagação para
frente
        perda ← média de −[w·y·log p + (1−y)·log(1−p)]
        calcula o gradiente da perda (backpropagation)
        atualiza os pesos com o otimizador (Adam/AdamW)
        a cada 10 lotes:
            perda_val ← perda da rede na validação
            se perda_val < melhor: melhor ← perda_val; salva checkpoint;
sem_melhora ← 0
            senão: sem_melhora ← sem_melhora + 1
            se sem_melhora = 30: encerra o treino
retorna a rede do melhor checkpoint
```

Justificativa. Captura interações não lineares entre atributos (ex.: valor alto só é arriscado combinado com prazo longo) e permite incorporar o custo assimétrico diretamente na função de perda.

Parâmetros principais. Camadas ocultas (capacidade do modelo), ativação (ReLU ou tanh), dropout 0,3 e weight decay 10^{-4} (regularização, importante numa base de 600 exemplos de treino), otimizador Adam ou AdamW com taxa de aprendizado 0,001, e parada antecipada (paciência de 30 avaliações) para evitar overfitting.

Configurações testadas (8): ativação {ReLU, tanh} × otimizador {Adam, AdamW} × camadas ocultas {[16], [32, 16]}.

Pré-processamento específico: log + padronização (redes convergem mal com entradas em escalas muito diferentes) e peso 2,33 para a classe mau na perda.

2. KNN — k vizinhos mais próximos (scikit-learn)

Funcionamento. Não há treino propriamente dito: o modelo guarda os clientes de treino. Para um novo cliente, calcula a distância euclidiana até todos eles, seleciona os k mais próximos e estima P(mau) como a proporção de maus entre esses vizinhos. Com ponderação por distância, vizinhos mais próximos pesam mais (peso = 1 / distância).

```
entrada: treino (X, y), cliente x, número de vizinhos k
para cada cliente i do treino: d_i ← distância(x, X_i)
V ← os k clientes com menor d_i
se ponderado: P(mau) ← Σ_{i∈V} y_i / d_i  ÷  Σ_{i∈V} 1 / d_i
senão:        P(mau) ← (nº de maus em V) / k
```

Justificativa. Método simples e interpretável ("clientes parecidos têm comportamento parecido"), que serve de referência para os modelos mais complexos.

Parâmetros principais. k (pequeno = sensível a ruído; grande = fronteira mais suave) e ponderação dos votos (uniforme ou por distância).

Configurações testadas (6): k {5, 15, 31} × ponderação {uniforme, distância}.

Pré-processamento específico: log + padronização são obrigatórios (sem eles o valor do crédito, na casa dos milhares, dominaria a distância). O KNN não aceita peso de classe; o desbalanceamento é tratado só pelo limiar.

3. Random Forest (scikit-learn)

Funcionamento. Treina muitas árvores de decisão, cada uma em uma amostra bootstrap do treino (sorteio com reposição) e escolhendo, a cada nó, a melhor divisão entre um subconjunto aleatório de atributos. As árvores ficam diferentes entre si, e a média das previsões reduz a variância que uma árvore única teria.

```
entrada: treino (X, y) com n clientes e p colunas, número de árvores B
para b = 1 até B:
    D_b ← sorteia n clientes do treino com reposição (bootstrap)
    árvore_b ← ConstruirArvore(D_b)

ConstruirArvore(D):
    se o nó tem menos de 2·min_samples_leaf clientes ou atingiu max_depth ou
é puro:
        retorna uma folha com a proporção (ponderada) de maus em D
    sorteia m = √p colunas
    escolhe, entre elas, a coluna e o limiar que mais reduzem a impureza de
Gini
    divide D em esquerda/direita e chama ConstruirArvore em cada parte

previsão: P(mau | x) ← média de P_b(mau | x) nas B árvores
```

Justificativa. Lida bem com a mistura de atributos categóricos e numéricos, captura interações sem precisar de normalização, é robusto a outliers e costuma funcionar bem em bases pequenas sem muito ajuste.

Parâmetros principais. 300 árvores (suficiente para estabilizar a média), $\sqrt{p}$ atributos por nó (diversidade entre árvores), profundidade máxima e mínimo de clientes por folha (controlam o overfitting de cada árvore) e peso 2,33 para a classe mau.

Configurações testadas (4): profundidade máxima {sem limite, 8} × mínimo por folha {1, 5}.

Pré-processamento específico: sem log e sem padronização; one-hot nos categóricos.

4. Gradient Boosting (scikit-learn, HistGradientBoostingClassifier)

Funcionamento. Treina árvores pequenas **em sequência**. Cada nova árvore é ajustada aos erros (gradiente da perda logística) do conjunto atual, ou seja, aprende a corrigir o que as anteriores erraram. A contribuição de cada árvore é reduzida pela taxa de aprendizado. A versão "Hist" agrupa os valores numéricos em faixas, o que acelera a busca das divisões.

```
entrada: treino (X, y), número de árvores M, taxa de aprendizado η
F_0(x) ← log(proporção de maus / proporção de bons)       # previsão inicial
para m = 1 até M:
    para cada cliente i: r_i ← y_i − sigmoide(F_{m−1}(x_i))   # resíduos =
gradiente negativo
    h_m ← árvore pequena ajustada para prever r_i (com peso maior nos maus)
    F_m(x) ← F_{m−1}(x) + η · h_m(x)
previsão: P(mau | x) ← sigmoide(F_M(x))
```

Justificativa. É em geral um dos algoritmos mais fortes para dados tabulares; por corrigir erros de forma incremental, consegue modelar relações complexas com árvores rasas.

Parâmetros principais. Taxa de aprendizado (menor = aprende devagar e generaliza melhor, mas precisa de mais árvores), profundidade máxima de cada árvore (complexidade das interações), 200 árvores, até 31 folhas e no mínimo 20 clientes por folha, peso 2,33 para a classe mau.

Configurações testadas (4): taxa de aprendizado {0,03; 0,1} × profundidade máxima {3, sem limite}.

Pré-processamento específico: sem log e sem padronização; one-hot nos categóricos.

IV. Resultados e discussão

O **Random Forest** teve o menor custo no teste (0,530 por cliente) e a maior AUC (0,777), seguido de perto pelo Gradient Boosting (0,555); o KNN foi o mais fraco (0,670). As diferenças entre Random Forest, Gradient Boosting e MLP, porém, são da ordem da variação entre divisões dos dados.

Tabela comparativa (teste)

Melhor configuração de cada algoritmo, escolhida pelo custo médio na validação; métricas no conjunto de teste (200 clientes, 60 maus).

Algoritmo	Configuração escolhida	Limiar	Acurácia	Precisão	Recall	Especificidade	F1	AUC	Custo médio
Random Forest	sem limite de profundidade, mín. 5 por folha	0,34	**0,610**	**0,427**	0,883	**0,493**	**0,576**	**0,777**	**0,530**
Gradient Boosting	taxa 0,03, profundidade 3	0,26	0,565	0,400	**0,900**	0,421	0,554	0,768	0,555
MLP	ReLU, AdamW, camadas [32, 16]	0,29	0,480	0,355	**0,900**	0,300	0,509	0,770	0,640
KNN	k = 15, ponderação por distância	0,14	0,490	0,356	0,867	0,329	0,505	0,713	0,670
Referência: negar todos	—	—	0,300	0,300	1,000	0,000	0,462	0,500	0,700

Matrizes de confusão (teste)

Algoritmo	VN (bons aprovados)	FP (bons negados)	FN (maus aprovados)	VP (maus negados)	Custo total (5·FN + FP)
Random Forest	69	71	7	53	106
Gradient Boosting	59	81	6	54	111
MLP	42	98	6	54	128
KNN	46	94	8	52	134

Todos os modelos deixam passar poucos maus pagadores (6 a 8 de 60), porque o limiar foi ajustado ao custo 5:1. **A diferença entre eles está nos bons clientes negados:** o Random Forest nega 71 de 140, a MLP nega 98. Ou seja, com o mesmo nível de detecção de maus, o Random Forest separa melhor os bons.

Efeito dos parâmetros

Algoritmo	Configuração	Custo médio (validação)	Custo médio (teste)	AUC (teste)
Random Forest	sem limite, mín. 5 por folha ← escolhida	0,475	0,530	0,777
Random Forest	profundidade 8, mín. 5 por folha	0,485	0,510	0,788
Random Forest	profundidade 8, mín. 1 por folha	0,510	0,535	0,792
Random Forest	sem limite, mín. 1 por folha	0,535	0,535	0,784
Gradient Boosting	taxa 0,03, profundidade 3 ← escolhida	0,515	0,555	0,768
Gradient Boosting	taxa 0,03, sem limite	0,535	0,590	0,765
Gradient Boosting	taxa 0,1, profundidade 3	0,565	0,605	0,762
Gradient Boosting	taxa 0,1, sem limite	0,625	0,625	0,779
MLP	ReLU, AdamW, [32, 16] ← escolhida	0,495	0,640	0,770
MLP	tanh, Adam ou AdamW, [32, 16]	0,500	0,640	0,770
MLP	ReLU, Adam, [32, 16]	0,505	0,675	0,770
MLP	tanh, Adam ou AdamW, [16]	0,505	0,585	0,776
MLP	ReLU, Adam ou AdamW, [16]	0,520	0,585	0,779
KNN	k = 15, distância ← escolhida	0,585	0,670	0,713
KNN	k = 15, uniforme	0,600	0,655	0,708
KNN	k = 31, distância	0,615	0,615	0,755
KNN	k = 31, uniforme	0,620	0,610	0,756

Algoritmo	Configuração	Custo médio (validação)	Custo médio (teste)	AUC (teste)
KNN	k = 5, distância	0,645	0,685	0,684
KNN	k = 5, uniforme	0,660	0,655	0,670

- **Gradient Boosting:** foi o mais sensível aos parâmetros. Taxa 0,1 com árvores sem limite de profundidade sobreajustou (pior custo, 0,625); a taxa menor com árvores rasas foi a melhor.
- **Random Forest:** exigir pelo menos 5 clientes por folha melhorou o custo nas duas profundidades, porque evita folhas que memorizam casos isolados. A variação total foi pequena (0,510 a 0,535 no teste): o Random Forest é pouco sensível aos parâmetros.
- **KNN:** k = 5 foi o pior (AUC ⩽ 0,684), por ser sensível a ruído. Aumentar k para 31 elevou a AUC de teste para 0,756, mas a validação escolheu k = 15. A ponderação por distância quase não mudou o resultado.
- **MLP:** Adam e AdamW deram resultados praticamente iguais (o weight decay de 10^{-4} é pequeno demais para diferenciá-los). A rede menor ([16]) teve melhor custo no teste (0,585), mas a validação preferiu [32, 16] por uma diferença de 0,010 a 0,025, menor que o ruído de uma validação de 200 clientes.

Nos quatro algoritmos a configuração melhor na validação nem sempre é a melhor no teste. Isso não indica erro: com cerca de 60 maus em cada conjunto, uma diferença de poucos clientes muda o ranking. Por isso medimos a variabilidade a seguir.

Influência do desbalanceamento

Repetimos a melhor configuração de cada algoritmo em três situações: sem nenhum tratamento (sem peso de classe, limiar 0,5), só com peso de classe, e com peso de classe mais limiar otimizado na validação.

Algoritmo	Tratamento	Recall (teste)	Especificidade (teste)	AUC (teste)	Custo médio (teste)
Random Forest	nenhum	0,367	0,914	0,783	1,010
Random Forest	peso de classe	0,633	0,779	0,777	0,705
Random Forest	peso + limiar otimizado	0,883	0,493	0,777	0,530
Gradient Boosting	nenhum	0,517	0,857	0,762	0,825
Gradient Boosting	peso de classe	0,683	0,700	0,768	0,685

Algoritmo	Tratamento	Recall (teste)	Especificidade (teste)	AUC (teste)	Custo médio (teste)
Gradient Boosting	peso + limiar otimizado	0,900	0,421	0,768	0,555
MLP	nenhum	0,350	0,929	0,772	1,025
MLP	peso de classe	0,733	0,700	0,770	0,610
MLP	peso + limiar otimizado	0,900	0,300	0,770	0,640
KNN	nenhum	0,300	0,936	0,713	1,095
KNN	limiar otimizado	0,867	0,329	0,713	0,670

O desbalanceamento influenciou fortemente os resultados. Sem tratamento, os modelos detectam só 30% a 52% dos maus e custam **mais que negar crédito a todos** (0,83 a 1,10 contra 0,70), embora tenham acurácia mais alta. O peso de classe e o limiar **não mudam a AUC** (a capacidade de ordenar os clientes), apenas o ponto de operação: trocam bons negados por maus detectados, o que compensa com custo 5:1. Na MLP, o limiar otimizado na validação piorou o teste (0,610 → 0,640), outro efeito do tamanho reduzido da validação.

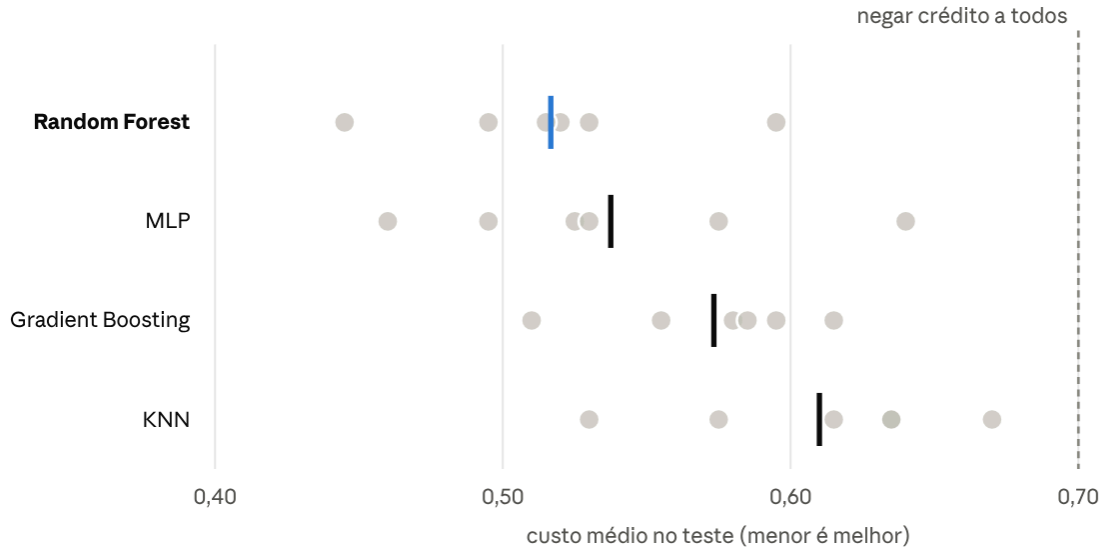
Variabilidade entre divisões

Rodamos a configuração escolhida de cada algoritmo em 6 divisões aleatórias diferentes (a principal e mais 5), sempre com o mesmo protocolo.

Algoritmo	Custo médio (teste), média ± desvio	AUC (teste), média ± desvio	Recall, média	F1, média	Menor custo em
Random Forest	**0,517 ± 0,049**	**0,793 ± 0,035**	0,914	0,573	4 de 6 divisões + 1 empate
MLP	0,538 ± 0,063	0,787 ± 0,025	0,881	0,577	1 de 6 + 1 empate
Gradient Boosting	0,573 ± 0,037	0,769 ± 0,026	0,867	0,558	0 de 6
KNN	0,610 ± 0,050	0,719 ± 0,025	0,894	0,527	0 de 6

Random Forest teve o menor custo médio nas 6 divisões testadas

Custo médio por cliente no teste, (5·FN + FP) / n. Cada ponto é uma divisão dos dados; o traço vertical é a média.



resultados/extra/robustez_seeds.csv + divisão principal · configuração escolhida de cada algoritmo · 6 divisões de 200 clientes de teste

O Random Forest manteve o menor custo médio e a maior AUC média. A MLP fica próxima na média, mas varia mais (desvio 0,063). O KNN é consistentemente o mais fraco. A ordem MLP × Gradient Boosting se inverte em relação à divisão principal, o que confirma que diferenças menores que ~0,05 de custo não são conclusivas com 200 clientes de teste.

Respostas às questões do enunciado

- **Qual algoritmo teve o melhor desempenho? Depende da medida?** O Random Forest, em custo, F1, AUC, acurácia, precisão e especificidade. Em recall, Gradient Boosting e MLP empatam à frente (0,900 contra 0,883), mas à custa de negar muito mais bons clientes. Sem tratamento do desbalanceamento, a acurácia apontaria os modelos de pior custo, então a resposta depende sim da medida.
- **A alteração dos parâmetros modificou os resultados?** Sim, principalmente no Gradient Boosting (custo de 0,555 a 0,625) e no KNN (AUC de 0,670 a 0,756). Random Forest e MLP foram pouco sensíveis.
- **Os algoritmos erraram de forma diferente?** Os erros graves (maus aprovados) foram quase os mesmos, 6 a 8 em todos. A diferença está nos bons negados: 71 no Random Forest contra 94 a 98 em KNN e MLP. O KNN precisou de um limiar muito baixo (0,14) porque suas probabilidades são grosseiras (proporções entre 15 vizinhos).
- **O desbalanceamento influenciou a avaliação?** Sim: sem tratamento, todos os modelos custaram mais que negar crédito a todos, apesar de acurácia entre 74,5% e 75,5%. Por isso usamos custo, recall, F1 e AUC em vez de acurácia.

- **Que características da base explicam as diferenças?** Os atributos são majoritariamente categóricos (13 de 20) e nenhum separa bem as classes sozinho, o que favorece árvores, que combinam regras do tipo "conta negativa e prazo longo". O KNN sofre com as 59 colunas do one-hot, em que muitas distâncias ficam parecidas. A base pequena (600 clientes de treino) limita a MLP e penaliza modelos que sobreajustam, como o Gradient Boosting com taxa alta.
- **Qual modelo recomendamos?** O **Random Forest** (sem limite de profundidade, mínimo de 5 clientes por folha, peso de classe e limiar ≈ 0,34). Ele teve o menor custo e a maior AUC na divisão principal e na média de 6 divisões, foi pouco sensível aos parâmetros, não exige normalização e permite medir a importância de cada atributo, o que ajuda a explicar decisões de crédito. Na prática, ele detecta cerca de 9 em cada 10 maus pagadores, ao custo de negar crédito a metade dos bons, um compromisso aceitável apenas com a matriz de custo 5:1; se o banco valorizar mais os bons clientes, basta recalibrar o limiar.

V. Conclusões

O Random Forest, com peso de classe e limiar ajustado ao custo, é o modelo recomendado: custo médio de 0,530 no teste principal e 0,517 ± 0,049 em 6 divisões, contra 0,700 de negar crédito a todos.

Principais aprendizados.

- Em problemas desbalanceados com custos assimétricos, a acurácia engana: os modelos sem tratamento tiveram a maior acurácia (~75%) e o pior custo.
- Peso de classe e ajuste do limiar não tornam o modelo "mais inteligente" (a AUC não muda); eles escolhem o ponto de operação certo para o negócio.
- O pré-processamento deve seguir o algoritmo: padronização é essencial para KNN e MLP e irrelevante para árvores.
- Evitar vazamento de dados exige disciplina: ajustar escalas, codificação, limiar e escolha de modelo só com treino e validação, e usar o teste uma única vez.
- Em base pequena, um modelo mais complexo (MLP) não superou um conjunto de árvores bem configurado.

Limitações.

- Validação e teste têm 200 clientes (~60 maus). Diferenças menores que ~0,05 de custo não são conclusivas, e o ranking entre MLP e Gradient Boosting mudou entre divisões.
- Checkpoint, limiar e escolha da configuração usam a mesma validação, o que torna as métricas de validação otimistas e, às vezes, escolhe uma configuração que não é a melhor no teste.

- A grade de parâmetros foi pequena (4 a 8 configurações por algoritmo).
- A base é de 1994, da Alemanha, e contém atributos sensíveis (sexo/estado civil e nacionalidade); não avaliamos viés do modelo contra esses grupos.
- As decisões de pré-processamento (ex.: aplicar log no valor) vieram da análise exploratória feita sobre a base inteira, embora os parâmetros das transformações tenham sido aprendidos só no treino.

Possíveis melhorias.

- Validação cruzada estratificada repetida (ou validação cruzada aninhada) para escolher parâmetros e estimar o desempenho com menos variância.
- Busca de hiperparâmetros mais ampla (ex.: busca aleatória ou bayesiana) e um limiar escolhido em partição separada.
- Calibração das probabilidades e análise de importância dos atributos (ex.: SHAP) para explicar as decisões.
- Avaliação de justiça por grupo e teste do modelo sem os atributos sensíveis.

VI. Referências

Dados

1. HOFMANN, H. *Statlog (German Credit Data)*. UCI Machine Learning Repository, 1994. DOI: 10.24432/C5NC77. Disponível em: archive.ics.uci.edu/dataset/144.

Algoritmos

2. BREIMAN, L. Random Forests. *Machine Learning*, v. 45, n. 1, p. 5–32, 2001.
3. FRIEDMAN, J. H. Greedy function approximation: a gradient boosting machine. *Annals of Statistics*, v. 29, n. 5, p. 1189–1232, 2001.
4. KE, G. et al. LightGBM: a highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, v. 30, 2017. (base do HistGradientBoostingClassifier)
5. COVER, T.; HART, P. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, v. 13, n. 1, p. 21–27, 1967.
6. RUMELHART, D. E.; HINTON, G. E.; WILLIAMS, R. J. Learning representations by back-propagating errors. *Nature*, v. 323, p. 533–536, 1986.
7. KINGMA, D. P.; BA, J. Adam: a method for stochastic optimization. *International Conference on Learning Representations (ICLR)*, 2015.
8. LOSHCHILOV, I.; HUTTER, F. Decoupled weight decay regularization. *International Conference on Learning Representations (ICLR)*, 2019.

Bibliotecas e ferramentas

9. PEDREGOSA, F. et al. Scikit-learn: machine learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011. scikit-learn.org
10. PASZKE, A. et al. PyTorch: an imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, v. 32, 2019. pytorch.org
11. McKINNEY, W. Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 2010. (pandas)
12. HARRIS, C. R. et al. Array programming with NumPy. *Nature*, v. 585, p. 357–362, 2020.
13. VIRTANEN, P. et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, v. 17, p. 261–272, 2020.
14. HUNTER, J. D. Matplotlib: a 2D graphics environment. *Computing in Science & Engineering*, v. 9, n. 3, p. 90–95, 2007.
15. ANTHROPIC. *Claude Code* (modelo Claude Opus 5.5). Ferramenta de IA generativa usada conforme a declaração abaixo, 2026.

Declaração de uso de Inteligência Artificial Generativa

O grupo utilizou IA generativa na implementação do código, na execução dos experimentos e na redação de uma primeira versão deste relatório.

Ferramenta. Claude Code (Anthropic), extensão para o VS Code, com o modelo Claude Opus 5.5.

Finalidade e etapas em que foi usada.

Etapa do trabalho	Uso da ferramenta
Leitura do enunciado e obtenção da base	Leitura do PDF e download da base da UCI
a. Exploração dos dados	Script de análise exploratória (estatísticas, testes, gráficos, tabelas)
b. Pré-processamento	Implementação das transformações decididas a partir da análise
c. Algoritmos	Implementação da MLP em PyTorch (configurável por YAML), do KNN, do Random Forest e do Gradient Boosting
d. Planejamento experimental	Protocolo treino/validação/teste, prevenção de vazamento de dados, script de comparação, experimentos de desbalanceamento e de variabilidade

Etapa do trabalho	Uso da ferramenta
Investigação	Análise de por que a AUC de teste superava a de validação
Relatório	Primeira versão do texto deste documento

Prompts utilizados (transcritos como foram enviados, em ordem):

1. "faça o trabalho espeficicado em Trabalho-CD.pdf. o dataset do meu trabalho é 10. quero fazer uma MLP simples para prever o dataset, mas primeiro quero que você faça um script para realizar uma analise exploratoria nos dados, apos obtermos esses dados, vamos construir a nossa MLP. para a nossa mlp, use pytorch. deve haver um arquivo yaml de configuração. deve haver a opção de rodar na cpu ou gpu, o optimizer que queremos rodar, função de ativação escolhida, função de loss, etc... a ideia é poder testar varias abordagens com facilidade. leia o pdf, baixe o dataet e vamos começar com o script de analise exploratoria de dados"
2. "melhore os graficos, deixe apenas graficos mais simples de compreender. já pode fazer o modelo."
3. "deixe na configuração a parte de divisão de split treino eval e test. cada linha deve ser selecionada aleatoriamente. muito cuidado com data leakage. o treino deve salvar sempre o melhor checkpoint, ou seja, o que possuir o melhor score na metrica de validação escolhida. deixe uma opção tambem, por exemplo, se o modelo não melhor nos proximos x steps de eval, finalizamos o treino. remova isso: momentum: 0.9 # usado por sgd e rmsprop nesterov: false # só sgd, deixe só adam e adamw. de scheduler, remova isso: factor: 0.5 # plateau patience: 10 # plateau e aoção de plateau. de loss, remova isso focal_gamma: 2.0 # só focal e a opção de focal. demais configs q não fizeram mais sentido apos essas remoções, pode remover tambem, e não se esqueça de tirar o codigo e revisar todo o projeto apos isso"
4. "nosso script tem wieght bias para ajudar no dentro de classes desbalanceadas??"
5. "mude uma coisa, mude no nome para weight bias. e me explique essa parte:" (seguido do trecho `evaluation` do arquivo de configuração: limiar, custos e métrica principal)
6. "não é escolher o melhor experiomento pela teste, é escolher pelo split de validação"
7. "ajuste oq não fizer sentido. além disso, organize melhor o projeto. separe bem a analise exploratoria e o modelo em sí. não se esqueça de atualizar a documentação"
8. "exclua os experimentos prontos e rode denovo, estou achando estranho o test estar com auc mais q a validação, investigue isso"
9. "para concluirmos o trabalho, precisamos de mais 3 algoritmos para rodarmos na base de dados, qual vocÊ sugere?"

10. "faça a implementação dos outros 3 algoritmos, não exagere, faça os de forma simples, mas bem feita. deixei tudo organizado de forma com que possamos comparar o resultados de cada algoritmo depois" — complementado por: "esqueci de falar, na regressão logistica, use knn", "os outros devem ser random forest e gradint boosting" e "ao inves de sklearn_models, faça um diretorio novo para cada uma das 3 abordagens"
11. "agora faça utro documento respondendo tudo o que se pede no trabalho em Trabalho-CD.pdf"

Como as respostas foram verificadas.

- **Verificações automáticas no código:** o protocolo confere que nenhuma linha aparece em dois conjuntos (treino, validação, teste) e registra um identificador da divisão, para garantir que todos os algoritmos usaram as mesmas linhas.
- **Verificação independente das execuções:** um script separado recarregou cada modelo salvo e confirmou, para as 23 execuções da comparação principal, que (i) o pré-processamento foi ajustado só no treino, (ii) o limiar é reproduzido usando apenas a validação e (iii) as métricas de teste são reproduzidas exatamente a partir do modelo salvo. Essa verificação encontrou e permitiu corrigir um erro real (a métrica de custo era tratada como "maior é melhor" na escolha do checkpoint da MLP).
- **Investigação de resultado suspeito:** a AUC de teste maior que a de validação foi investigada com regressão logística de controle, troca dos papéis validação/teste, intervalos de confiança bootstrap e 30 divisões diferentes; concluímos que era variação da divisão, não vazamento.
- **Gráficos** foram inspecionados visualmente e refeitos quando confusos; números citados neste texto foram conferidos contra os arquivos de resultados.
- **[Grupo: descrever aqui as verificações feitas pelos integrantes — por exemplo, execução dos scripts, leitura e entendimento do código, conferência das fórmulas e dos pseudocódigos, revisão e reescrita do texto.]**

O uso da ferramenta não substitui o entendimento do trabalho: os integrantes revisaram o código, os resultados e o texto e devem ser capazes de explicar as decisões metodológicas e os resultados apresentados.